

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: MLA0303

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO4: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr. DUMPALA PRASANTH

QUESTIONS: 10

Total Marks: 50

Annexure A

Mock Interview

Duration: 1hr

1. A machine learning engineer applies Reinforcement Learning to solve sequential decision-making problems. Explain how an agent learns optimal behavior through interaction with the environment. **(5 Marks)**

2. If you are asked to design a Reinforcement Learning solution for a real-time system such as ride-sharing or delivery optimization, explain how you would formulate the problem using a Markov Decision Process. **(5 Marks)**

3. Compare model-free and model-based Reinforcement Learning approaches. Explain their working principles and how they handle uncertainty. **(5 Marks)**

4. A recommendation system using Reinforcement Learning is underperforming. Explain how reward sparsity and hyperparameter tuning can be diagnosed and improved. **(5 Marks)**

5. If you are implementing an RL-based game-playing AI, describe how you would design the learning pipeline and ensure training stability. **(5 Marks)**
6. A company wants to use Reinforcement Learning for dynamic pricing. Explain how the agent learns from interaction and improves pricing decisions over time. **(5 Marks)**
7. If an RL model is not converging properly, explain how you would identify and fix issues related to learning rate and exploration strategy. **(5 Marks)**
8. A robotics system uses Reinforcement Learning for navigation. Explain how the environment and agent interaction leads to optimal path learning. **(5 Marks)**
9. If you are asked to improve an RL model with delayed rewards, explain techniques to handle credit assignment problems. **(5 Marks)**
10. A financial firm uses Reinforcement Learning for trading decisions. Explain how uncertainty and risk can be handled in the learning process. **(5 Marks)**

Code of Conduct:

*I **K Uday Kiran Reddy(192425211)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:kuday

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

Q1. A machine learning engineer applies Reinforcement Learning to solve sequential decision-making problems. Explain how an agent learns optimal behavior through interaction with the environment.

Reinforcement Learning (RL) is a learning approach in which an agent learns how to make sequential decisions by interacting with an environment. At each time step, the agent observes the current state, selects an action according to its policy, receives a reward, and moves to a new state.

The basic interaction is:

State $S \rightarrow$ Agent/Policy $\rightarrow$ Action $A \rightarrow$ Environment $\rightarrow$ Reward R and Next State $S' \rightarrow$ Agent

The agent attempts to maximize the cumulative reward rather than only the immediate reward. A discounted return can be written as:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

where γ is the discount factor.

Initially, the agent may not know which actions are best. It explores different actions and learns from the rewards obtained. Over repeated interactions, it updates a policy, value function, or action-value function. A policy $\pi(a|s)$ specifies which action should be selected in a given state.

The learning cycle is:

1. Observe state.
2. Select an action.
3. Execute the action.
4. Receive reward and next state.
5. Update knowledge.
6. Repeat until the policy becomes effective.

For example, in a delivery system, an agent can receive positive rewards for fast, successful deliveries and negative rewards for delays. Over time it learns routes and decisions that increase total reward.

Thus, an RL agent learns optimal behavior through repeated trial, feedback, exploration, and policy improvement.

Q2. If you are asked to design a Reinforcement Learning solution for a real-time system such as ride-sharing or delivery optimization, explain how you would formulate the problem using a Markov Decision Process.

A real-time ride-sharing or delivery problem can be formulated as a Markov Decision Process (MDP). An MDP is represented by:

$$\text{MDP} = (\text{S}, \text{A}, \text{P}, \text{R}, \gamma)$$

where S is the state space, A is the action space, P is the transition probability, R is the reward function, and γ is the discount factor.

For a ride-sharing system:

1. State (S):

The state can contain the driver's current location, passenger locations, available drivers, traffic condition, pending requests, time of day, and vehicle status.

2. Action (A):

The agent may assign a driver to a passenger, accept/reject a request, choose a route, or reposition an available driver.

3. Transition (P):

After an action, the system moves to a new state because passengers are served, new requests arrive, traffic changes, or drivers change location.

4. Reward (R):

Positive reward can be given for successful and fast service. Negative reward can represent long waiting time, unnecessary travel, cancellations, or high operating cost.

5. Discount factor (γ):

γ controls the importance of future rewards. A value close to 1 gives more importance to long-term performance.

The Markov property means that the relevant current state contains the information needed to predict the next state distribution after an action.

Example objective:

Maximize total reward = service quality – waiting cost – travel cost.

The RL agent interacts with the live system, observes new requests and conditions, takes decisions, receives rewards, and improves its policy.

Therefore, the MDP gives a structured mathematical representation that allows RL to solve dynamic ride-sharing or delivery decisions.

Q3. Compare model-free and model-based Reinforcement Learning approaches. Explain their working principles and how they handle uncertainty.

Model-Free RL and Model-Based RL differ mainly in whether the agent explicitly learns or uses a model of environment dynamics.

Model-Free RL:

The agent directly learns a value function, action-value function, or policy from experience. It does not explicitly construct a model that predicts future states.

Working:

State $\rightarrow$ Action $\rightarrow$ Reward/Next State $\rightarrow$ Update Policy or Value

Examples include Q-Learning, SARSA, and policy-gradient methods.

Model-Based RL:

The agent learns or uses an environment model that describes how states and rewards change after actions. It can then use the model for simulation and planning before choosing actions.

Working:

Real experience $\rightarrow$ Learn model $\rightarrow$ Predict future outcomes $\rightarrow$ Plan $\rightarrow$ Select action

Comparison:

- Model-free: simpler model structure, but often requires substantial real interaction.
- Model-based: can use planning and simulated experience, but model learning introduces additional complexity.
- Model-free methods learn directly from observed consequences.
- Model-based methods can evaluate possible futures using their learned model.

Handling uncertainty:

In model-free RL, uncertainty can be handled through exploration strategies such as ϵ -greedy exploration, stochastic policies, or uncertainty-aware value estimates. In model-based RL, uncertainty can be represented in transition or reward predictions and considered during planning.

For example, a delivery system may use model-free RL to learn which dispatch decisions produce good rewards, while a model-based approach can predict how traffic and driver movements may change after a dispatch decision.

In summary, model-free RL learns what to do directly, whereas model-based RL learns how the environment behaves and can use that knowledge for planning.

Q4. A recommendation system using Reinforcement Learning is underperforming. Explain how reward sparsity and hyperparameter tuning can be diagnosed and improved.

Two important causes of poor RL performance in a recommendation system are sparse rewards and unsuitable hyperparameters.

Reward sparsity means that useful feedback is received infrequently. For example, a recommendation system may receive a strong reward only when a user clicks, watches, purchases, or completes an item. Many interactions may produce zero or weak reward. This makes it difficult for the agent to determine which earlier decisions contributed to the eventual outcome.

Diagnosis of reward sparsity:

- Inspect the percentage of zero/near-zero rewards.
- Examine reward distribution and reward frequency.
- Check how long it takes between an action and meaningful feedback.
- Monitor learning curves and policy behavior.

Improvements:

- Design informative intermediate rewards where appropriate.
- Use reward shaping carefully so that intermediate behavior supports the actual objective.
- Use suitable discounting so future feedback is retained.
- Improve exploration so the system can discover rewarding recommendations.

Hyperparameter tuning:

Important parameters include learning rate, discount factor, exploration parameters, batch size, network size, and update frequency, depending on the RL algorithm.

Diagnosis:

- Compare training runs using controlled experiments.
- Monitor reward, loss, policy stability, and validation performance.
- Check whether the learning rate is too high or too low.
- Check whether exploration becomes too weak too early.

Improvement:

Use systematic search such as grid search, random search, or controlled trial experiments.

Tune one group of parameters at a time and evaluate on held-out scenarios.

Q5. If you are implementing an RL-based game-playing AI, describe how you would design the learning pipeline and ensure training stability.

An RL game-playing AI can be organized as a pipeline from game-state observation to action selection, experience collection, learning, evaluation, and policy improvement.

Pipeline:

Game Environment → State Observation → Policy/Agent → Action → Game Result/Reward
→ Experience Storage → Training → Updated Policy

Step 1: Define the state.

Represent the game board, player status, legal actions, score, and other information required for decision-making.

Step 2: Define actions.

The action space contains all legal moves available to the agent.

Step 3: Design rewards.

Use rewards for winning, losing, achieving objectives, or useful intermediate events. Rewards should encourage the intended game behavior.

Step 4: Select the RL algorithm.

Depending on the action space and task, use a suitable value-based or policy-based method.

Step 5: Collect experience.

Run games and store useful transitions such as (S, A, R, S') .

Step 6: Train the agent.

Sample training data and update policy or value parameters.

Training stability can be improved by:

- Appropriate learning rate.
- Controlled exploration.
- Reward normalization or careful reward scaling when appropriate.
- Experience replay for applicable algorithms.
- Target networks for applicable value-based deep RL methods.
- Batch training and stable update schedules.
- Monitoring reward, loss, and evaluation performance.
- Saving checkpoints and comparing against fixed evaluation conditions.

A separate evaluation process should measure win rate, average score, and consistency without excessive exploration.

Q6. A company wants to use Reinforcement Learning for dynamic pricing. Explain how the agent learns from interaction and improves pricing decisions over time.

Dynamic pricing can be formulated as an RL problem in which the agent repeatedly selects prices and observes the resulting customer and business response.

State:

The state can include current demand, inventory, time period, historical sales, customer activity, competitor conditions, and other relevant business variables.

Action:

The action is the price or a permitted pricing level/range.

Reward:

The reward can represent profit, revenue, sales volume, customer retention, or a weighted business objective. Penalties can be included for undesirable outcomes such as excessive inventory or unacceptable customer effects.

Interaction:

State $\rightarrow$ Price Action $\rightarrow$ Market Response $\rightarrow$ Reward + Next State

The agent initially explores different pricing decisions. It observes how customers respond and records the resulting rewards. The policy or value function is updated using this experience.

Over time:

1. Observe current demand and inventory.
2. Select a price using the current policy.
3. Observe sales and business response.
4. Calculate reward.
5. Update the policy/value estimate.
6. Repeat.

The discount factor can make future profit important rather than focusing only on immediate revenue.

A practical system should also impose business constraints, such as allowed price ranges and safety rules. Offline evaluation or simulation can be used before deployment to reduce risk.

Thus, the agent gradually learns pricing decisions that improve the selected long-term business objective while adapting to changing demand.

Q7. If an RL model is not converging properly, explain how you would identify and fix issues related to learning rate and exploration strategy.

If an RL model is not converging, learning rate and exploration should be investigated systematically.

Learning rate:

The learning rate controls the size of parameter updates.

If it is too high:

- Updates can overshoot useful solutions.
- Reward may fluctuate strongly.
- Training can become unstable.

If it is too low:

- Learning can become extremely slow.
- The agent may appear stuck for a long time.

Diagnosis:

Monitor training reward, loss, parameter changes, and evaluation performance across multiple runs. Compare smaller and larger learning rates under otherwise controlled conditions.

Fix:

Try a smaller learning rate when updates are unstable. Try a larger rate when learning is excessively slow. Learning-rate schedules can also be used when appropriate.

Exploration:

Exploration allows the agent to try actions that are not currently considered best. Exploitation selects actions believed to be best.

With insufficient exploration:

The agent can get stuck in a poor policy because it does not discover better actions.

With excessive exploration:

The agent may fail to exploit what it has learned and performance can remain unstable.

For ϵ -greedy exploration:

- With probability ϵ , choose a random action.
- Otherwise choose the current best action.

A common approach is to start with higher exploration and gradually reduce ϵ as learning progresses, while maintaining enough exploration for the problem.

A systematic procedure is:

1. Establish a baseline.
2. Change learning rate independently.
3. Compare reward curves.
4. Tune exploration schedule.
5. Repeat across multiple random seeds.
6. Select settings that provide stable evaluation performance.

Therefore, convergence problems should be diagnosed from learning behavior rather than changing many parameters at the same time.

Q8. A robotics system uses Reinforcement Learning for navigation. Explain how the environment and agent interaction leads to optimal path learning.

In an RL-based robotic navigation system, the robot is the agent and the physical or simulated workspace is the environment.

State:

The state may contain the robot's position, orientation, sensor observations, nearby obstacles, target location, and motion information.

Actions:

The robot can move forward, turn left, turn right, stop, or choose a permitted motion command.

Reward:

The robot can receive positive reward for reaching the destination and negative rewards for collisions, excessive path length, delays, or unsafe movement.

Interaction loop:

Robot observes state → selects movement → environment changes → receives reward and new state → updates policy → repeats

During training, the robot explores different paths. Successful paths receive better cumulative returns. Unsafe or inefficient paths receive lower returns.

The objective can be represented as:

Maximize cumulative reward = goal achievement – collision penalty – path/time cost.

For a navigation problem, the value function estimates the long-term quality of being in a state or taking an action. The policy gradually learns actions that lead toward high-value states.

Training can initially be performed in simulation to allow large amounts of interaction and reduce physical risk. The learned policy can then be evaluated and, when appropriate, transferred to the physical robot with safety constraints.

Thus, repeated interaction and reward feedback allow the robot to learn a policy that favors safe and efficient routes to the target.

Q9. If you are asked to improve an RL model with delayed rewards, explain techniques to handle credit assignment problems.

A delayed reward occurs when an important reward is received long after the actions that caused it. The credit assignment problem is determining which earlier actions deserve credit or blame for the final outcome.

For example, in a game, an agent may make many moves before winning. The final winning reward must influence the earlier decisions that contributed to success.

Techniques to address delayed rewards include:

1. Discounted returns:

Use a return such as $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$ so that future rewards can influence earlier decisions.

2. Reward shaping:

Introduce meaningful intermediate rewards when they correctly encourage progress toward the real objective. This should be designed carefully to avoid changing the intended task.

3. Eligibility traces:

Methods such as $TD(\lambda)$ distribute learning information across a sequence of recently visited states/actions, helping earlier decisions receive credit.

4. Advantage/value estimation:

Estimate how much better an action was compared with the expected value of the state. This can improve learning signals for policy-based methods.

5. Multi-step returns:

Use several future rewards rather than only the immediate reward to propagate information backward more quickly.

6. Experience replay where applicable:

Past transitions can be revisited during learning, allowing delayed reward information to influence earlier experiences.

The main objective is to propagate useful information from delayed outcomes to the decisions that contributed to those outcomes.

Therefore, delayed-reward RL should use return estimation and credit-assignment mechanisms that connect long-term outcomes with earlier actions.

Q10. A financial firm uses Reinforcement Learning for trading decisions. Explain how uncertainty and risk can be handled in the learning process.

Financial trading contains substantial uncertainty because market prices, demand, volatility, liquidity, and external events can change unpredictably. An RL system therefore should not optimize only expected return; it should also account for risk and uncertainty.

State:

The state may include market prices, technical indicators, portfolio holdings, cash position, volatility measures, and other permitted market information.

Actions:

The agent may buy, sell, hold, or select a portfolio allocation subject to constraints.

Reward:

Instead of using raw profit alone, the reward can incorporate transaction costs, drawdown, volatility, or other risk measures. A general form is:

$$\text{Risk-adjusted reward} = \text{Return} - \lambda \times \text{Risk Cost}$$

where λ controls the importance assigned to risk.

Handling uncertainty:

- Use stochastic policies or exploration mechanisms.
- Evaluate performance across different market conditions.
- Use uncertainty-aware model estimates when a model-based approach is used.
- Avoid excessive reliance on a single historical pattern.
- Test the policy on data or scenarios not used for training.

Risk management:

- Impose position and exposure limits.
- Include transaction costs and slippage in evaluation.
- Penalize excessive drawdowns or undesirable volatility.

- Use stop/trading constraints where required by the application.
- Monitor performance continuously.

The training process should distinguish exploration from real deployment. Risky exploratory actions should not be allowed to violate operational constraints.

Thus, a financial RL system should combine reward optimization with explicit risk controls, uncertainty analysis, and robust evaluation rather than simply maximizing historical profit.